Beyond Task Completion: Auditing Verification and Conformance in Tool-Evolving Agents

Anonymous Author(s)

Abstract

Agents that synthesize their own tools ship a second artifact alongside each answer: a software library that future tasks reuse, compose, and depend on. Task completion (TC) certifies the answer; it does not certify the library. We show that this matters. In a preserved-source trace from a separate pilot, one synthesized tool drives session TC to 0.95 while failing every held-out input for the capability it claims to implement – a verification-vs.-conformance gap a pass-rate benchmark cannot see.

EvolveTool-Bench is built to make that gap measurable. Sessions are structured into seed, gap, variant, composition, regression, and adversarial roles; runs emit per-tool manifests, verified-subset TC, correct-vs.-incorrect reuse, and audit traces designed to be re-emitted as a post-deployment monitoring schema. In a verified-subset Haiku 4.5 pilot over five protocols (3 seeds, 8 sessions, 51 verified decisions per pass), TC alone does not separate the protocols; the audit layer does. One pre-specified contrast shows unvalidated synthesis can *reduce* verified completion (-7.1 pp, CI $[-13.9, -0.1]$).

The strict pilot does not yet preserve per-tool source, so we illustrate the verification-conformance gap rather than measure it in-regime; aligning per-tool conformance under the strict pilot is the framework's next artifact-hardening step. We release the harness, verifier definitions, and reproducibility manifests, and argue that an audit schema of this shape – verifier coverage, reuse decomposition, held-out conformance – is the minimum a deployed tool-evolving agent should emit between releases.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → **Software verification and validation**.

Keywords

agentic AI, tool use, evaluation, benchmark, auditability, verification

1 Introduction

A growing class of agents does more than call a fixed tool set. These systems write Python functions, API clients, parsers, analysis scripts, or richer skill bundles, and then carry those artifacts forward into later tasks. VOYAGER learns reusable code skills in Minecraft [Wang et al.(2023)]; CREATOR separates tool creation from tool use [Qian et al.(2023)]; LLMs as Tool Makers and ToolMaker study agents that construct tools for other agents to call [Cai et al.(2023), Wölflein et al.(2025)]; EvoSkill and CoEvoSkills treat skills as evolving units of agent knowledge [Alzubi et al.(2026), Zhang et al.(2026)]. In all of these settings, the agent leaves behind a software artifact.

A task-completion score tells us whether the answer was accepted; it does not tell us what the agent left behind. A generated library can solve the next task while still containing brittle parsing logic, redundant functions, or behavior changes that silently break earlier capabilities. For deployed or long-running agents, those differences are not cosmetic: they are the difference between a one-off success and an auditable system.

EvolveTool-Bench is built around this distinction. It evaluates agents that may create tools during a session and asks not only "did the task pass?" but also "what happened to the library?" Each session has known dependency structure: seed tasks test provided tools, gap tasks invite new capabilities, variant tasks test reuse, composition tasks test chaining, regression tasks revisit prior behavior, and adversarial tasks probe edge cases. The resulting traces let a researcher inspect whether a failure came from synthesis, reuse, verification, composition, or regression.

The benchmark design contains 99 tasks across three domains. Of these, the data-transformation and numerical domains currently expose deterministic task verifiers; the API-orchestration session is part of the benchmark design but excluded from the main TC denominator until its verifiers are complete. Unverified tasks are reported separately and not credited as successes. This narrows the empirical claim of any single submission while tightening the evaluation claim of the framework as a whole.

Our contributions are:

(1) **A diagnostic protocol for evolving tool libraries.** Sessions are structured so that creation, reuse, composition, regression, and adversarial behavior can be inspected separately.
(2) **Artifact-level measurements beyond TC.** We define and log tool creation, raw reuse, correct reuse, incorrect reuse, utilization, composition, and regression signals alongside verified task completion.
(3) **Verifier-aware reporting.** The strict analysis separates the benchmark's full design scope from the currently verified subset, preventing unverified tasks from inflating task-completion claims.
(4) **An audit-trace view of agent evaluation.** Runs preserve task outcomes, tools created, tools used, reuse outcomes, and verifier status so that aggregate scores can be traced back to concrete artifacts.

The headline empirical finding is that TC alone fails to separate the protocols on the verified-subset Haiku pilot, while the audit layer does (§5); one pre-registered contrast also surfaces a directional cost of unvalidated synthesis. The headline

conceptual finding is the verification-vs.-conformance gap, which we define and illustrate with a preserved-source trace from a separate pilot: a tool can pass every in-session verifier call and still fail every held-out input for the capability it claims to implement. The strict-pilot harness does not yet preserve per-tool source, so the gap is illustrated rather than measured in-regime; closing that loop is the framework's next artifact-hardening step. Together these argue that benchmarks for tool-evolving agents must make the state of the tool library visible – not only the pass rate.

2 Related Work

Tool and skill creation. Systems such as CREATOR, LLMs as Tool Makers, ToolMaker, EvoSkill, and CoEvoSkills all study agents that create tools or skills rather than merely selecting from a fixed tool set [Qian et al.(2023), Cai et al.(2023), Wölflein et al.(2025), Alzubi et al.(2026), Zhang et al.(2026)]. Their evaluations typically focus on downstream success, per-tool validation, or skill-level pass rates. EvolveTool-Bench is complementary: it asks what happens to the library over time, especially under reuse, composition, and regression pressure.

Agent and tool-use benchmarks. ToolBench and AgentBench evaluate tool-using agents with predefined environments or APIs [Qin et al.(2023), Liu et al.(2024)]. VendingBench stresses long-horizon agent behavior with a fixed operational environment [Backlund and Petersson(2025)]. These benchmarks are valuable, but the tool inventory is generally not an artifact produced by the agent. Our setting differs because the generated library is part of the system under evaluation.

Skill benchmarks and library learning. SkillsBench shows that skills can help some tasks and hurt others, especially when the skills are self-generated [Li et al.(2026)]. Program-synthesis work such as DreamCoder studies learned libraries and evaluates their effect on solve rates or compression [Ellis et al.(2021)]. The question we address sits between these traditions: when an LLM agent accumulates executable artifacts, can we tell whether the resulting library is correct, reusable, and stable?

Quality of generated code. Tool-Genesis, NoFunEval, and related code-evaluation work measure compliance, unit-test performance, maintainability, or non-functional requirements for generated code [Xia et al.(2026), Singhal et al.(2024)]. EvolveTool-Bench borrows the software-engineering lens but applies it at session and library granularity. A single generated function can look reasonable in isolation while still making the library harder to use.

3 Benchmark Design

3.1 Sessions and task roles

A session is a fixed sequence of 11 tasks with shared library state. The sequence is intentionally simple: it creates controlled opportunities for a system to use, create, reuse, compose, and preserve tools. Table 1 summarizes the task roles.

Table 1: Task roles in each session. The fixed order lets the harness attribute behavior to creation, reuse, composition, regression, or edge-case handling rather than reporting one undifferentiated pass rate.

Role	Count	Diagnostic purpose
Seed	3	Use provided tools; establish baseline behavior.
Gap	2	Solve a task requiring a missing capability.
Variant	2	Recognize that a prior capability can be reused.
Compose	1	Chain multiple capabilities or tools.
Regress	1	Revisit an earlier contract after the library has changed.
Adversarial	2	Handle edge cases or malformed inputs.

The benchmark contains three domains and nine sessions: five data-transformation sessions, one API-orchestration session, and three numerical-computation sessions. Data-transformation sessions use proprietary formats such as binary encodings and schema-like validators. Numerical sessions use curve fitting, signal processing, and constrained optimization tasks. The API session uses a mock service with HMAC authentication and cursor handling. The full benchmark contains 99 tasks, but the strict analysis in this paper uses only tasks with deterministic verifiers.

3.2 Verification policy

Verification is part of the contribution, not a footnote. A task contributes to strict TC only if it has an explicit expected output or a custom verifier; tasks without deterministic verification are reported separately, never credited because the answer was long or plausible. This avoids the common failure mode in agent benchmarks where format-looking output is mistaken for correctness.

The API-orchestration session is excluded from the main strict TC analysis because its task verifiers are not yet complete. The verified-subset pilot therefore covers 8 sessions and 51 verified task decisions per complete system pass. The API session remains in the benchmark design as an important tool-use domain, but is not used to support the numerical claims in this submission.

Table 2 shows how the 51-task verified subset is distributed across task roles. Seed tasks carry no graded outcome by design: they exist to expose the agent to the provided library before any creation, so they have no expected output or verifier. The diagnostic-critical roles – gap, variant, regress – are near-fully covered (93.8–100%); compose and adversarial are partially covered. The verified subset therefore exercises the roles the framework leans on for its claims, while remaining honest about where verifier work is still needed.

Table 2: Verifier coverage of the strict subset by task role (eight verified-subset sessions; API-orchestration excluded). Seed tasks deliberately carry no graded outcome and are not part of the TC denominator. The roles that the diagnostic framework leans on – gap, variant, regress – are near-fully covered; compose and adversarial have partial coverage.

Role	Verified	Total	Coverage
Seed (warm-up, ungraded)	0	24	—
Gap	15	16	93.8%
Variant	15	16	93.8%
Compose	4	8	50.0%
Regress	8	8	100.0%
Adversarial	9	16	56.2%
Strict-subset total (graded)	51	64	79.7%

3.3 Metrics and audit trace

Verified task completion (TC).. TC is the fraction of verified tasks whose outputs satisfy the task verifier. It measures user-visible success on the verified subset: necessary but not sufficient.

Tool creation and reuse. Before each task, the harness records the current library; after the task, it records newly created tools and tools used. Reuse is counted when a task invokes a tool that existed before the task began. We decompose reuse into:

- **correct reuse**: a prior tool was used and the task passed;
- **incorrect reuse**: a prior tool was used and the task failed.

We report the *correct-reuse rate* as the fraction of reuse calls that coincide with a passing task; we avoid the term "precision" because no false-positive denominator is defined. This distinction is important because raw reuse can mean either useful abstraction or repeated reliance on a defective artifact.

Library-health diagnostics. The broader framework also supports redundancy, utilization, composition, regression stability, and capability-aligned hidden conformance tests. These diagnostics are part of the benchmark design and audit schema. The verified-subset submission reports only the signals aligned with the current verified manifests: TC, tools created, correct reuse, incorrect reuse, and the correct-reuse rate. A composite library-health ranking is held back until the conformance-test alignment is complete (Supplement §E reports it on a separate Sonnet pilot where source preservation made the alignment possible).

Audit trace. Each run emits a per-task record so aggregate scores can be traced to concrete artifacts. Figure 1 shows the schema for one task record from `data_transform_s1`: a reader can move from any cell in Table 3 to the task, seed, tool, and pass/fail decision that produced it.

```
{"task_id": "data_transform_s1/gap_1",
"role": "gap", "verifier_status": "verified",
"model": "claude-haiku-4-5", "seed": 0,
"outcome": "pass",
"tools_available_before": ["parse_header", ...],
"tools_used": ["decode_abr_format"],
"tools_created": ["decode_abr_format"],
"reuse_outcome": null,
"artifact_summary": {"loc": 47, "imports": ["base64"]},
"llm_calls": 3}
```

Figure 1: One audit-trace record (schema, redacted). The same schema is the unit of post-deployment monitoring discussed in §6.

4 Experimental Setup

The strict pilot evaluates five representative protocols. They are not meant to exhaust the design space; they provide enough variation to test whether the harness can distinguish tool-creation and reuse behavior.

(1) **No-Evolution.** The system receives the seed tools and never modifies the library. It is the lower-bound protocol for library evolution.
(2) **One-Shot.** When the system attempts tool creation, it adds a generated function without iterative repair or external validation. This tests unvalidated creation.
(3) **EvoSkill-style.** A strategy-level adaptation inspired by EvoSkill. It maintains natural-language skill hints rather than executable tools.
(4) **ToolMaker-style.** A protocol adaptation inspired by ToolMaker's diagnose-and-rewrite loop. It is not a native port; the original setting assumes reference tests and repository inputs that our harness does not provide.
(5) **CREATOR-style.** A decision/execution split inspired by CREATOR, where the system explicitly decides whether a tool should be created before executing the task.

All verified-subset runs use Claude Haiku 4.5 (snapshot `claude-haiku-4-5-20251001`) via the Anthropic API with temperature 1.0 and the model default top-p. Each protocol is executed with three integer seeds (`0, 1, 2`) over eight verified-subset sessions, producing 24 session rows per protocol. Synthesised tools execute in an in-process sandbox with a five-second wall-clock limit; harness state resets between sessions so library accumulation is per-session, not cross-session. The same Anthropic API key, decoding parameters, task stream, and verifier set are used for all protocols; only the tool-creation control flow varies. We scope this pilot to a single model snapshot on purpose: varying the model would conflate protocol effects with model-evolution effects – the latter is a separate evaluation question the same harness is built to ask, and we surface it in Supplement §E as a same-schema-different-model probe.

Pre-registration. The three contrasts in Table 4, the bootstrap parameters (B=10,000, seed `20260604`), the BH correction, and the verified-subset definition were specified before

Table 3: Strict verified-subset pilot. TC is computed only on tasks with deterministic verifiers; unverified tasks are reported separately and excluded from the TC denominator. Means and standard errors are over 24 session rows (3 seeds × 8 verified-subset sessions) using Claude Haiku 4.5.

System	TC	SE	Tools	Correct-reuse rate
One-Shot	0.358	0.077	114	0.333
No-Evolution	0.337	0.064	0	0.333
EvoSkill-style	0.332	0.062	0	0.250
ToolMaker-style	0.292	0.054	18	0.375
CREATOR-style	0.287	0.048	111	0.333

running the Haiku pilot, and are stored in `claims.json` and `statistical_report.json` in the released artifact. No contrasts were added, removed, or redirected after results were observed. The decision to report verifier coverage, reuse decomposition, and audit traces was part of the framework design, not a response to the TC outcome.

Statistical procedure. For each pre-specified contrast we form per-session paired differences across the 24 verified-subset session rows (3 seeds × 8 sessions), bootstrap-resample those pairs B=10,000 times with seed `20260604`, and report the bootstrap point estimate and 95% percentile CI. The three pre-specified contrasts in Table 4 are Benjamini–Hochberg adjusted at α=0.05. A contrast is marked *supported* only if its CI lies strictly above zero in the pre-specified positive direction. Per-session outcomes are largely seed-stable in this pilot, so the effective number of independent units sits between 8 and 24; the bootstrap resamples (seed, session) pairs without assuming seed-independence.

Artifacts. The benchmark sources, harness, verifier definitions, canonical manifests (`run_manifest.jsonl`, `tool_manifest.jsonl`, `aggregate.json`, `audit_report.json`, `statistical_report.json`, `claims.json`), and the scripts that regenerate every table in this paper from disk are included in the anonymised supplement and will be released under an open licence on acceptance.

5 Results

5.1 TC alone does not separate the protocols

Verified-subset TC differences are small relative to per-session standard errors and cross-session heterogeneity. Across the five protocols TC ranges from 0.287 (CREATOR-style) to 0.358 (One-Shot), with No-Evolution (0.337) and EvoSkill-style (0.332) in between (Table 3).

The pre-specified improvement tests do not support a positive TC claim (Table 4). One-Shot is not distinguishable from No-Evolution in the expected positive direction. ToolMaker-style does not improve over One-Shot. The Creator − One-Shot contrast is the only pre-specified comparison whose 95% CI excludes zero: −7.1 pp with CI [−13.9, −0.1], just brushing zero from below. The CI is consistent with – but does not establish – a synthesis-loop cost in the adapted-baseline regime. We do not draw a method claim about the original CREATOR design from an adapted protocol; we report this only as a hypothesis-generating signal worth a powered follow-up.

Table 4: Pre-specified TC contrasts in the strict verified-subset pilot. Positive effects were expected; none support a TC-improvement claim. Values are absolute TC differences.

Contrast	Δ	95% CI	Improves?
OneShot − NoEvol	0.0208	[−0.0347, 0.0729]	No
Creator − OneShot	-0.0710	[−0.1389, −0.0012]	No
ToolMaker − OneShot	-0.0655	[−0.1443, 0.0174]	No

Power. With 24 paired session rows and the observed within-session standard deviation of TC (≈ 0.30), and accounting for the seed-stability above, the verified-subset pilot is powered (80%, α=0.05) to detect paired TC differences of roughly 8–13 pp depending on whether effective n is treated as 24 or as 8 sessions. The largest observed $|\Delta|$ in Table 4 is 7.1 pp. The null TC result should therefore be read as "no large protocol effect at this scale" rather than "no effect." Smaller true effects – below ∼ 7 pp – remain compatible with these CIs and motivate the larger multi-model pilots that the released harness is built to support.

5.2 The audit view separates what TC does not

The lack of a TC winner is precisely when the audit layer matters most. One-Shot and CREATOR-style create many tools (114 and 111 respectively), while No-Evolution and EvoSkill-style create none. ToolMaker-style creates fewer tools (18), consistent with a more conservative validation loop. These differences are invisible in a TC-only table. The reuse denominators in Table 3 are non-zero even for the zero-tools systems: reuse counts include calls to provided seed tools (the seed library each session starts from), not only to tools the agent itself created.

The correct-reuse rate – the fraction of reuse calls that coincide with a passing task – adds a second axis. One-Shot has 18 correct vs. 36 incorrect reuses out of 54 (0.33), No-Evolution 9/18 out of 27 (0.33), EvoSkill-style 9/27 out of 36 (0.25), CREATOR-style 12/24 out of 36 (0.33), and ToolMaker-style 9/15 out of 24 (0.38). At this scale the differences are not powered to discriminate between protocols, and we report them only for trace inspection and protocol design. What they show is the kind of decomposition the harness exposes: when a system uses prior artifacts, does that reuse coincide with success or with failure?

5.3 A verification-vs.-conformance gap that TC cannot see

A tool can pass every in-session verifier call and still fail every held-out input for the same capability – and TC will not see it. The verifier asks whether one particular call produced an acceptable output; a capability-aligned conformance suite asks whether the tool implements the capability on inputs the agent never saw. These two questions routinely disagree: a tool that structurally matches the example the agent used to write itself will pass the verifier and fail held-out conformance. That gap is what motivates the per-tool conformance layer the framework defines, and it is the failure mode an audit schema for a deployed tool-evolving agent has to surface.

Illustrative trace. We make the gap concrete with a preserved-source pilot from a separate run.[1] In `data_transform_s1`, the system synthesises one tool, `decode_abr_format`, for the `abr_binary_decode` gap task. Five subsequent tasks in the same session (variant, compose, regress, two adversarials) can reuse it. The tool's agent-written self-tests pass at C=0.50; in-session reuse splits one correct and one incorrect, with the incorrect reuse on the regress task.

When the same source is replayed against a held-out conformance suite for the capability – six unit inputs and six adversarial inputs the agent never saw – per-tool correctness is C=0.00: every held-out input fails. The tool passes in-session verifier calls by structurally matching its own examples; it does not implement the capability. This is the audit signal the framework is built to surface, and the conceptual reason the verified-subset TC numbers should be read as a floor on tool quality, not as a measure of it.

5.4 Verifier coverage is a first-class benchmark property

If unverified tasks are credited by a length or plausibility heuristic, absolute TC can be inflated and protocol comparisons move for the wrong reason. Restricting the main analysis to deterministic verifiers sacrifices benchmark coverage but buys measurement credibility. The cost is clear: the API-orchestration domain is not part of the verified-subset TC analysis. The framework already identifies what to do next: complete API verifiers, align hidden tests by capability, and regenerate the same tables from the full manifest. Adversarial probes are part of the design (Table 2 reports 56.2% coverage). The current pilot records pass/fail at session level rather than per role, so per-role adversarial pass rates are not isolated in this submission; closing that gap is part of the same coverage-and-trace work as the API verifiers.

[1]The trace below comes from a preserved-source Sonnet pilot, included as design-motivating evidence rather than as a verified-subset result; the Haiku pilot in Tables 3–4 records session/system summaries only. Adding per-tool source preservation to the strict harness so the same trace can be produced in-regime is the next artifact-hardening step (Supplement §D).

6 Discussion

Design lessons for tool-evolving agents. Tool creation alone is not evidence of progress: many tools can be created without improving verified task completion. Reuse needs a quality signal; raw reuse is ambiguous without correct/incorrect decomposition. Verifier coverage is a first-class benchmark property – a tool-evolving system is only as trustworthy as the tests and records used to decide what it has learned. Generated tools are persistent objects that can be reused, duplicated, retired, or accidentally made into load-bearing dependencies for later failures; a benchmark that ignores this treats every run as independent and loses the audit trail the deployment setting needs.

Model evolution as audit signal. The same harness, manifests, and verifier set apply equally to a Sonnet pilot reported in Supplement §E, where per-tool source preservation made the capability-aligned conformance layer computable. Library-Health contrasts that are statistically supported under Sonnet are not powered under Haiku, not because the framework fails but because synthesis quality at the smaller model is below the floor where reuse decomposition becomes informative. The audit schema thus doubles as a model-evolution monitor: re-emitting the same schema after a model swap reveals whether tool quality has tracked the model, and the comparison is the exact "model evolution and API risk" question that motivates monitoring deployed agents.

From benchmark to post-deployment monitor. We see the audit schema as a CI artifact, not only a research instrument. A tool-evolving agent shipped behind an API can re-emit `run_manifest.jsonl` and `tool_manifest.jsonl` on every release; a regression gate can then require that the correct-reuse rate and capability-aligned hidden-test conformance do not drop below the prior release on a frozen task slice. Per-task audit overhead is dominated by the LLM calls themselves – manifest emission, reuse decomposition, and per-tool source dumping add millisecond-scale latency and bounded disk I/O, so the layer is cheap enough to leave on in production. Concretely, three monitoring signals fall out of the schema with no extra instrumentation: (i) *silent rot* – session TC stays high while held-out conformance for a re-used capability drops between releases; (ii) *reuse trap* – raw reuse rises but the correct-reuse rate falls, indicating an emerging defective dependency; (iii) *regression delta* – a regress-role task fails after the same capability's gap-role task passes, identifying which release crossed the regression. Each signal is a thresholded check that can sit behind a release gate.

7 Limitations

Verified subset. The strict statistical analysis covers 8 sessions and 51 verified task decisions per system pass, not the full 99-task benchmark. The API-orchestration session is excluded from main TC until deterministic verifiers are complete.

Pilot scale and single-model setting. The verified-subset pilot uses one model (Claude Haiku 4.5) and three integer seeds; per-session outcomes are partly seed-stable so standard errors mainly reflect session heterogeneity. A smaller model is where tool-creation protocols are most stress-tested, so the null TC result is consistent with synthesis quality at Haiku scale being below the floor where protocol differences become powered. The pilot should be read as exposing what the framework decomposes and what minimum effect size it can rule out (§5), not as a verdict on tool-creation protocols in general. Multi-model runs are the natural next step the released harness is built for, and are the lens through which the model-evolution monitoring story in §6 can be operationalized.

Protocol adaptations. EvoSkill-style, ToolMaker-style, and CREATOR-style are adaptations to a shared open-task harness, not native reproductions of the original systems. The original CREATOR and ToolMaker pipelines assume reference test suites or repository-grounded inputs that our open synthesis tasks do not supply; the `-style` suffix is deliberate, and the paper does not treat any underperformance in this pilot as a verdict on the original methods.

Hidden conformance tests. The benchmark design includes capability-aligned hidden tests for tool conformance. The main strict-subset results in this submission do not yet use a fully aligned conformance manifest, so we do not headline per-tool correctness numbers. Completing this alignment is the next artifact-hardening step.

Composite library-health score. A composite library-health score (or *EvolveTool-Score*) is useful as a dashboard but premature until conformance and verifier maps are complete; the current submission reports decomposed metrics.

8 Conclusion

Tool-evolving agents create a new evaluation target: the library they leave behind. A task-completion score is necessary but not sufficient. EvolveTool-Bench provides a structured way to inspect this artifact through task roles, verifier-aware TC, reuse decomposition, capability-aligned conformance, and trace preservation. The verified-subset pilot does not support a TC-improvement claim for any tested protocol, but it makes the verification-vs.-conformance gap concrete and surfaces three thresholded monitoring signals that a deployed tool-evolving agent can re-emit between releases.

Reusable Evaluation Practices

We recommend that future benchmarks for tool-evolving agents:

(1) separate task success from generated-artifact behavior;
(2) report verifier coverage and exclude unverified tasks from TC claims;
(3) distinguish correct reuse from incorrect reuse;
(4) preserve generated artifacts, artifact summaries, and task/session traces;
(5) include regression and composition probes after library growth;
(6) evaluate library-management policies such as promotion, deduplication, and retirement.

References

[Alzubi et al.(2026)] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. 2026. EvoSkill: Automated Skill Discovery for Multi-Agent Systems. *arXiv preprint arXiv:2603.02766* (2026).

[Backlund and Petersson(2025)] Axel Backlund and Lukas Petersson. 2025. Vending-Bench: A Benchmark for Long-Term Coherence of Autonomous Agents. *arXiv preprint arXiv:2502.15840* (2025).

[Cai et al.(2023)] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large Language Models as Tool Makers. *arXiv preprint arXiv:2305.17126* (2023).

[Ellis et al.(2021)] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*.

[Li et al.(2026)] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[Liu et al.(2024)] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.

[Qian et al.(2023)] Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

[Qin et al.(2023)] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).

[Singhal et al.(2024)] Manav Singhal, Tushar Aggarwal, Abhijeet Awasthi, Nagarajan Natarajan, and Aditya Kanade. 2024. NoFunEval: Funny How Code LMs Falter on Requirements Beyond Functional Correctness. *arXiv preprint arXiv:2401.15963* (2024).

[Wang et al.(2023)] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Advances in Neural Information Processing Systems* (2023).

[Wölflein et al.(2025)] Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. 2025. LLM Agents Making Agent Tools. *arXiv preprint arXiv:2502.11705* (2025).

[Xia et al.(2026)] Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. 2026. Tool-Genesis: A Task-Driven Tool Creation Benchmark for Self-Evolving Language Agents. *arXiv preprint arXiv:2603.05578* (2026).

[Zhang et al.(2026)] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, et al. 2026. CoEvoSkills:

Self-Evolving Agent Skills via Co-Evolutionary Verification. *arXiv preprint arXiv:2604.01687* (2026).